



15

Why React Native?

Meta (formerly Facebook) created React Native to build its mobile applications. It was started as a hackathon project in the summer of 2013 inside Facebook and open sourced for everyone in 2015. The motivation to do so originated from the fact that React for the web was so successful. So, if React is such a good tool for UI development, and you need a native application, then why fight it? Just make React work with native mobile OS UI elements! Therefore, in the same year, Facebook divided React into two independent libraries, `React` and `ReactDOM`, and since then, React had to work only with interfaces and not care about where these elements will be rendered.

In this chapter, you'll learn about the motivations for using React Native to build native mobile web applications. Here are the topics that we'll cover in this chapter:

- What is React Native?

- React and JSX are familiar
- The mobile browser experiences
- Android and iOS: different yet the same
- The case for mobile web apps

Technical requirements

There aren't any technical requirements for this chapter since it is a brief conceptual introduction to React Native.

What is React Native?

Earlier in this book, I introduced the notion of a render target, the thing that React components render to. The render target is abstract as far as the React programmer is concerned. For example, in React, the render target can be a string, or it could be the **Document Object Model (DOM)**. Therefore, your components never directly interface with the render target because you can never make assumptions about where the rendering is taking place.

A mobile platform has **UI widget libraries** that developers can leverage to build apps for that platform. On Android, developers implement apps using **Java** or **Kotlin**, while, on iOS, developers implement **Objective-C** or **Swift** apps. If you want a functional mobile app, you're going to have to pick one.

However, you'll need to learn both languages, as supporting only one of two major platforms isn't realistic for success.

For React developers, this isn't a problem. The same React components that you build work all over the place, even on mobile browsers! Having to learn two more programming languages to build and ship a mobile application is cost- and time-intensive. The solution to this is to introduce a new React platform that supports a new render target: native mobile UI widgets.

React Native uses a technique that makes asynchronous calls to the underlying mobile OS, which calls the native widget APIs. There's a JavaScript engine, and the React API is mostly the same as React for the web. The difference is with the target; instead of a DOM, there are asynchronous API calls. The concept is visualized here:

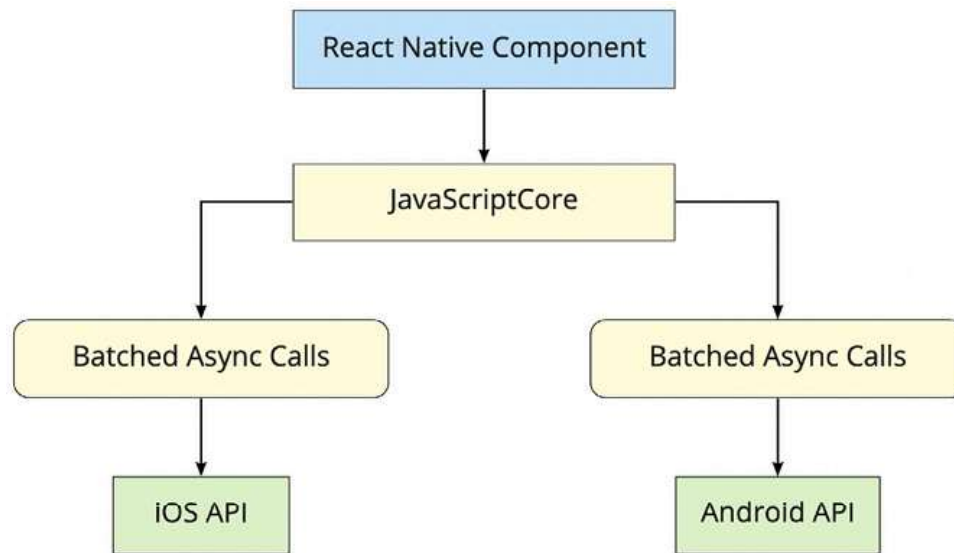


Figure 15.1: React Native workflow

This oversimplifies everything that's happening under the hood but the basic ideas are as follows:

- The same React library that's used on the web is used by **React Native** and runs in **JavaScriptCore**.
- Messages that are sent to native platform APIs are asynchronous and batched for performance purposes.
- React Native ships with components implemented for mobile platforms, instead of components that are HTML elements.
- React Native is just a way to render components via iOS and Android APIs. It can be replaced using the same concept with tvOS, Android TV, Windows, macOS, and even Web

again. This is reachable by **forks** and add-ons for React Native. In this part of the book, we will learn how to write mobile apps for iOS and Android. More information about other possible platforms can be found here:

<https://reactnative.dev/docs/out-of-tree-platforms>.

Much more on the history and mechanics of React Native can be found at

<https://engineering.fb.com/2015/03/26/android/react-native-bringing-modern-web-techniques-to-mobile/>

React and JSX are familiar

Implementing a new render target for React is not straightforward. It's essentially the same thing as inventing a new DOM that runs on iOS and Android. So, why go through all the trouble?

First, there's a huge demand for mobile apps. The reason is that the mobile web browser user experience isn't as good as the native app experience. Second, JSX is a fantastic tool for building UIs. Rather than having to learn new technology, it's much easier to use what you know.

It's the latter point that's the most relevant to you. If you're reading this book, you're probably interested in using React for both web applications and native mobile applications. I

can't put into words how valuable React is from a development-resource perspective. Instead of having a team that does web UIs, a team that does iOS, a team that does Android, and so on, there's just the UI team that understands React.

In the following section, you'll learn about the challenges of delivering good user experiences on mobile web browsers.

The mobile browser experiences

Mobile browsers lack many capabilities of mobile applications. This is because browsers cannot replicate the same native platform widgets as HTML elements. You can try to do this, but it's often better to just use the native widget rather than try to replicate it. This is partly because this requires less maintenance effort on your part, and partly because using widgets that are native to the platform means that they're consistent with the rest of the platform. For example, if a date picker in your application looks different from all the date pickers the user interacts with on their phone, this isn't a good thing. Familiarity is key and using native platform widgets makes familiarity possible.

User interactions on mobile devices are fundamentally different from the interactions that you typically design for the web. Web applications assume the presence of a mouse, for example, and that the click event on a button is just one phase.

However, things become more complicated when the user uses their fingers to interact with the screen. Mobile platforms have what's called a **gesture system** to deal with this. React Native is a much better candidate for handling gestures than React for the web because it handles these types of things that you don't have to think about much in a web app.

As the mobile platform is updated, you want the components of your app to stay updated, too. This isn't a problem with React Native because the app uses actual components from the platform. Once again, consistency and familiarity are important for a good user experience. So, when the buttons in your app look and behave in the same way as the buttons in every other app on the device, your app feels like part of the device.

Now that you understand what makes developing UIs for mobile browsers difficult, it's time to look at how React Native can bridge the gap between the different native platforms.

Android and iOS: different yet the same

When I first heard about React Native, I automatically thought that it would be some cross-platform solution that lets you write a single React application that will run natively on any device. However, the reality is more nuanced. While React

Native allows for a significant amount of code sharing between platforms, it's essential to understand that iOS and Android are different on many fundamental levels, and their user experience philosophies are different as well.

React Native's goal is to “learn once, write anywhere” rather than “write once, run anywhere.” This means that, in some cases, you'll want your app to take advantage of platform-specific widgets to provide a better user experience.

That being said, there have been advancements in the React Native ecosystem that enable more seamless cross-platform development.

For instance, **Expo** now supports web development, allowing you to run your app on the web using **React Native for Web**. This means that you can develop apps that work on Android, iOS, and the web using a single code base. Additionally, the **Tamagui UI kit** offers 100% support for both web and mobile platforms, making it easier to create apps that run on multiple platforms without sacrificing user experience.

In light of these developments, it's important to recognize that while React Native may not provide a perfect “write once, run anywhere” solution, it has come a long way in enabling more efficient cross-platform development. With tools like Expo and Tamagui, developers can create apps that work across differ-

ent platforms while still taking advantage of platform-specific features when necessary.

In the next section, we'll look at the case where mobile web apps that run in the browser might be a better fit for your users.

The case for mobile web apps

Not every one of your users is going to be willing to install an app, especially if you don't yet have a high download count and rating. The barrier to entry is much lower with web applications: the user only needs a browser.

Despite not being able to replicate everything that native platform UIs have to offer, you can still implement awesome things in a mobile web UI. Maybe having a good web UI is the first step toward getting those download counts and ratings up for your mobile app.

Ideally, what you should aim for is the following:

- Standard web (laptop/desktop browsers)
- Mobile web (phone/tablet browsers)
- Mobile apps (phone-/tablet-native platform)

Putting an equal amount of effort into all three of these spaces probably doesn't make much sense, as your users probably favor one area over another. Once you know, for example, that there's a high demand for your mobile app compared to the web versions, that's when you allocate more effort there.

Summary

In this chapter, you learned that React Native is an effort by Facebook to reuse React to create native mobile applications. React and JSX are good at declaring UI components, and since there's now a huge demand for mobile applications, it makes sense to use what you already know for the web.

The reason there's such a demand for mobile applications over mobile browsers is that they just feel better. Web applications lack the ability to handle mobile gestures the same way apps can, and they generally don't feel like part of the mobile experience from a look-and-feel perspective.

React Native has evolved significantly over the years, enabling developers to create more efficient cross-platform applications. While it's true that iOS and Android have fundamental differences, React Native has made strides in providing a more seamless development experience across platforms. However, it's important to remember that React Native's goal is to “learn once, write anywhere” rather than “write once, run any-

where.” This means that developers can still take advantage of platform-specific features to provide a better user experience.

Now that you know what React Native is and what its strengths are, you’ll learn how to get started with new React Native projects in the following chapter.